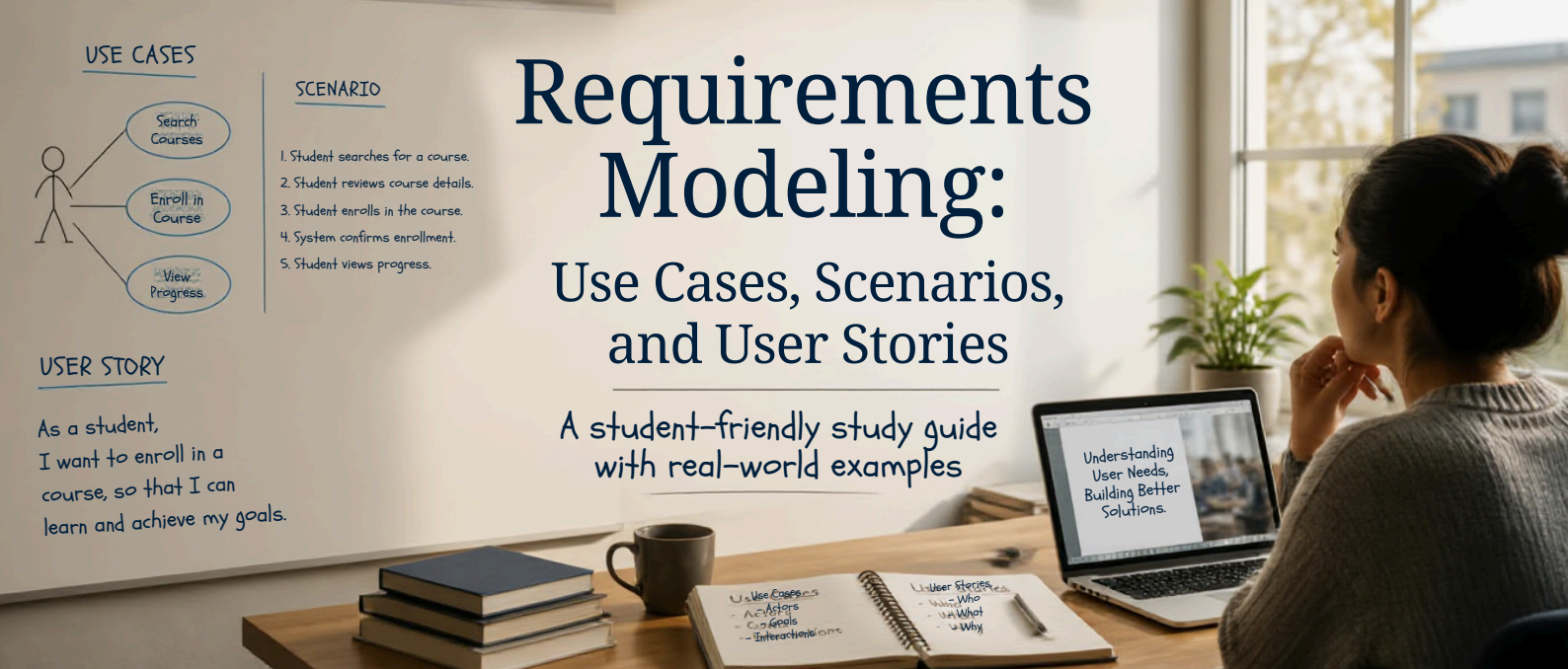
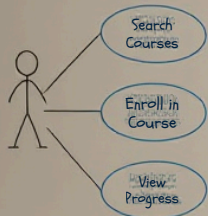




USE CASES



SCENARIO

1. Student searches for a course.
2. Student reviews course details.
3. Student enrolls in the course.
4. System confirms enrollment.
5. Student views progress.

USER STORY

As a student,
I want to enroll in a
course, so that I can
learn and achieve my goals.

Requirements Modeling:

Use Cases, Scenarios, and User Stories

A student-friendly study guide
with real-world examples

Requirements modeling is the process of describing **what a software system should do** (and what it should not do) in a way that both people and developers can understand.

It acts as a **bridge** between human needs (goals, tasks, policies) and technical specifications (features, rules, and tests). Good models reduce misunderstandings and help teams build the right system.

1) What is Requirements Modeling?

Requirements modeling turns ideas like “make it easy to book an appointment” into clear, testable statements such as “a patient can choose a doctor, pick an available time slot, and receive confirmation.”

You can think of it as answering four key questions:

- **Elicitation** — How do we **find** the requirements? (talk to users, observe work, read policies, run workshops)
- **Analysis** — What do these requirements **mean**, and do they conflict? (remove ambiguity, resolve contradictions, prioritize)
- **Specification** — How do we **write** the requirements clearly? (documents, use cases, user stories, diagrams, rules)
- **Validation** — Are we building the **right thing**? (review with stakeholders, prototypes, acceptance tests, check completeness)

Why this matters (exam-ready idea):

Requirements modeling can reduce defects because many defects originate from misunderstood or incomplete requirements.

Real-world example: Ride-sharing app

Imagine a team building a ride-sharing app. Without requirements modeling, someone might say: “Drivers should get jobs quickly.” That sounds good, but it is not specific.

With requirements modeling, the team makes it clear:

- **User goal** — Rider requests a trip and gets matched to a nearby driver.
- **Business rules** — Only approved drivers can accept rides; surge pricing applies in peak hours.
- **Edge cases** — No drivers available; payment fails; rider cancels after matching.

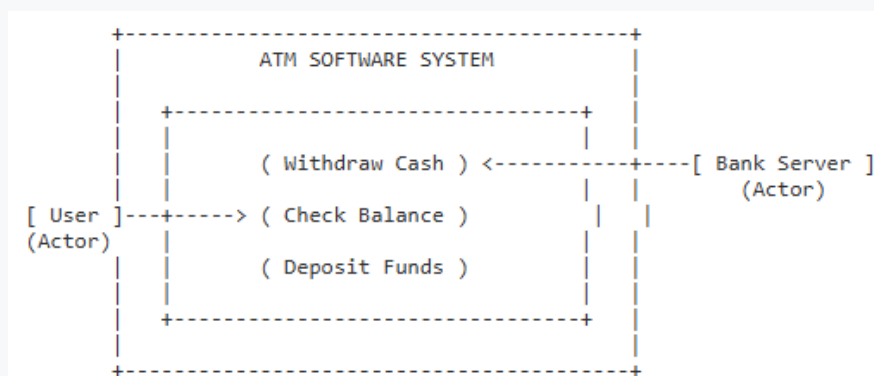
Now the team can design, implement, and test the system with fewer surprises.

2) Simple Use Case Diagram

A **use case diagram** shows:

- **Actors** — who uses the system (people or other systems).
- **Use cases** — the goals the actors achieve with the system (actions the system supports).

ATM example: Cash Withdrawal



What each part means

- **Customer (actor)** — the person using the ATM to get cash.
- **Bank Server (actor)** — an external system that verifies the card, PIN, and available funds.
- **Withdraw Cash (use case)** — the main goal: the customer receives cash.
- **Authenticate / Check Balance / Dispense Cash / Print Receipt** — smaller goals that support the main goal.

Real-world explanation:

The customer’s goal is simple: “I want cash.” The ATM cannot decide alone—it asks the bank server, “Is this card real? Is the PIN correct? Is there enough balance?” Only if the request is valid does the ATM dispense cash.

3) User Stories with Acceptance Criteria

User stories are short, simple descriptions of a feature written from the user's point of view. They are common in Agile projects because they are easy to discuss and easy to prioritize.

The basic format

- **As a** [type of user]
- **I want** [a goal / feature]
- **So that** [a reason / benefit]

Mental care example (healthcare system)

- **User story:** As a mental health nurse, I want to view a patient's medication history, so that I can avoid prescribing drugs that conflict with current treatments.

Acceptance criteria (Given-When-Then)

Acceptance criteria make the story **testable**. A common style is **Given-When-Then**:

- **Given** a starting situation (context)
- **When** an action happens
- **Then** an expected result must be true

1. **Given** a nurse is logged in and has permission to view the patient record,
2. **When** the nurse opens the patient's medication history,
3. **Then** the system shows current and past medications with dates and dosages.
4. **Given** the medication list includes a known allergy or contraindication flag,
5. **When** the nurse views the record,
6. **Then** the system highlights the warning clearly.

Additional example: Online shopping

- **User story:** As a shopper, I want to save items to a wishlist, so that I can buy them later or compare options.

Acceptance criteria (Given-When-Then):

1. **Given** I am signed in,
2. **When** I tap "Add to wishlist" on a product page,
3. **Then** the item appears in my wishlist immediately.
4. **Given** an item is in my wishlist,
5. **When** I remove it from the wishlist,
6. **Then** it no longer appears in the list and the wishlist count updates.

Study tip:

A user story explains **what** and **why**. Acceptance criteria explain **how we know it works** (they become test ideas).

4) The Role of Scenarios

A **scenario** is a step-by-step story showing how someone uses a system in a specific situation. Scenarios make requirements feel real because they describe what happens in order.

Normal flow vs. exception flow

- **Normal flow** — the “happy path” where everything goes as expected.
- **Exception flow** — what happens when something goes wrong (incorrect PIN, network down, item out of stock).

Example scenario: Withdrawing cash from an ATM

Normal flow

1. Customer inserts card.
2. ATM asks for PIN; customer enters PIN.
3. ATM asks bank server to authenticate and check balance.
4. Customer selects “Withdraw Cash” and enters an amount.
5. ATM dispenses cash and optionally prints a receipt.
6. ATM returns the card and ends the session.

Exception flows (examples)

- **Incorrect PIN** — After 3 failed attempts, the ATM keeps the card and ends the session.
- **Insufficient funds** — The bank server denies the request; ATM shows an error and asks for a smaller amount.
- **ATM out of cash** — The ATM cancels the withdrawal and suggests another ATM location.

How scenarios differ from use cases and user stories

- **Use case** — a structured view of interactions to achieve a goal (often covers multiple paths).
- **Scenario** — one concrete story, in order, for one situation (a specific path through a use case).
- **User story** — a short promise of value (“what” and “why”), usually paired with acceptance criteria for testing.

Memorable shortcut:

Use case = map of interactions, **scenario** = one route through the map, **user story** = why the trip matters.

5) Use Cases vs. User Stories

Use cases and user stories both describe requirements, but they shine in different situations. The table below makes the differences easy to spot.

Aspect	Use Cases	User Stories
Main purpose	Describe interactions between actors and the system to reach a goal.	Describe user value in one sentence (easy to discuss and plan).
Level of detail	Often more detailed (steps, alternate paths, exceptions).	Short and lightweight; details go into acceptance criteria and conversation.
Best for	Complex workflows, many rules, many exceptions (e.g., banking, healthcare).	Agile backlogs, incremental delivery, features that can be delivered in small pieces.
Typical output	Diagram + structured text (main flow + exception flows).	Story card + acceptance criteria (often Given–When–Then).
Quick example	“Withdraw cash” with steps and exceptions like wrong PIN and insufficient funds.	“As a customer, I want to withdraw cash, so that I can pay in cash today.”

When to use each (quick guide)

- **Choose a use case** when you need to understand a full workflow end-to-end, including errors and system responses.
- **Choose a user story** when you want a small, valuable feature you can plan, implement, and test in a short iteration.

6) Key Takeaways for Review

Plain-language summary

- **Requirements modeling** helps teams reduce defects by making needs clear before coding.
- **Use cases** capture interactions between actors and the system, including normal and exception flows.
- **User stories** support Agile work by expressing value simply, then adding testable acceptance criteria.
- **Scenarios** tell a step-by-step story and help everyone picture how the system is used.
- **Good requirements** are complete enough to build and **testable** (you can prove whether they are met).

Review checklist (before you submit requirements):

- ☐ Have we identified the main actors (people and other systems)?
- ☐ Do we understand the normal flow (happy path) from start to finish?
- ☐ Did we include key exception flows (common failures and edge cases)?
- ☐ Are user stories written with clear value (“So that...”)?
- ☐ Do acceptance criteria use Given–When–Then and stay testable?
- ☐ Can a teammate read this and implement it without guessing?

Memorable final example: Restaurant order

Imagine a self-service kiosk in a restaurant.

- **Use case** — “Place Order” (choose items, customize, pay, receive order number; handle payment failure or out-of-stock items).
- **Scenario** — “A student orders a burger meal, swaps fries for salad, pays by card, and gets order #42.”
- **User story** — “As a customer, I want to customize my meal, so that I can avoid ingredients I don’t eat.”

If you can model the restaurant kiosk clearly, you can model an ATM, a clinic system, or a ride-sharing app the same way—by turning goals into clear, testable requirements.